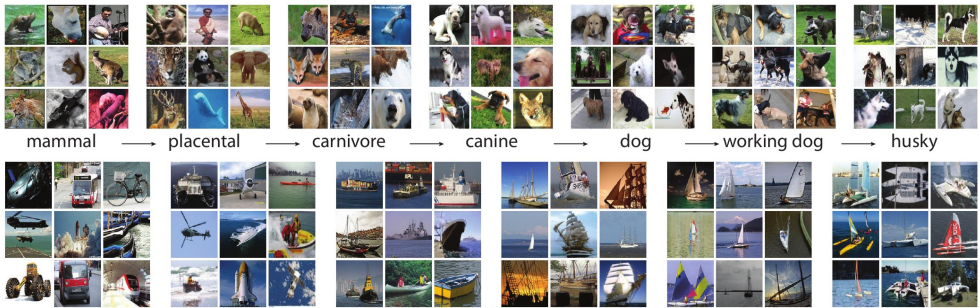
CNN Architectures

Ten years that changed computer vision

Parts of this chapter adapt LMU I2DL (slds-lmu), CC BY 4.0.

ImageNet: 14 million photos, sorted like a family tree

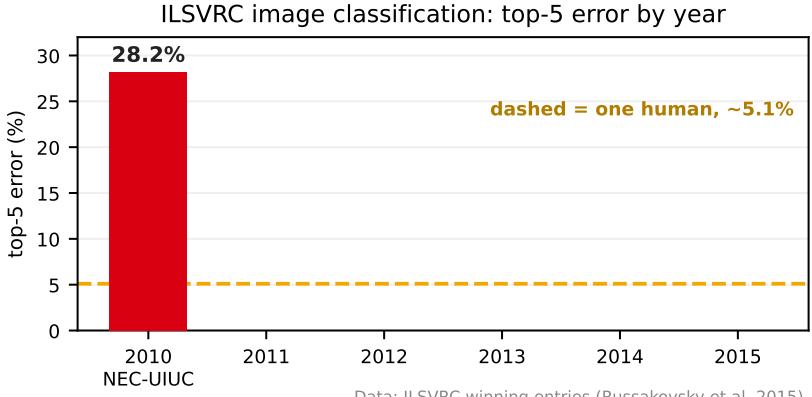


Two branches of the hierarchy, 9 random photos per category (Deng et al., "ImageNet", CVPR 2009, Fig. 1).

22,000 categories, taken from WordNet's tree of English nouns - from "mammal" all the way down to "husky". The yearly challenge used **1,000** of them, and **120 are dog breeds**: telling a Siberian husky from a malamute counts.

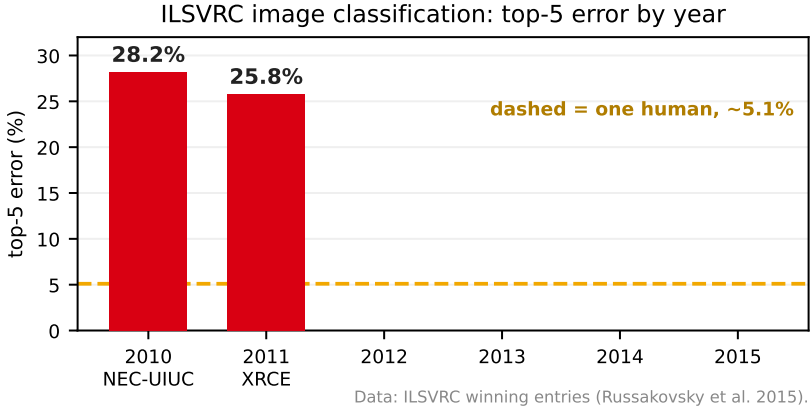
Until 2011 the best programs got about **1 photo in 4 wrong** - even when allowed **five guesses**. Then something happened.

One chart, one question



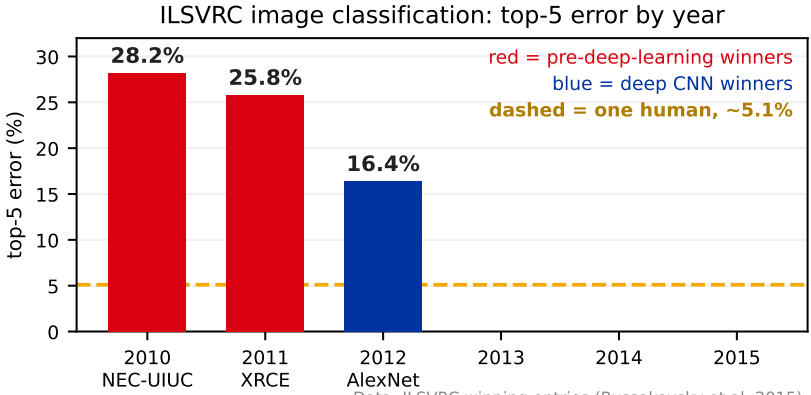
2010, NEC-UIUC (NEC Labs, Univ. of Illinois): hand-designed features (SIFT, LBP), encoded into one long vector, then linear SVMs. Experts designed the features - only the classifier learned.

One chart, one question



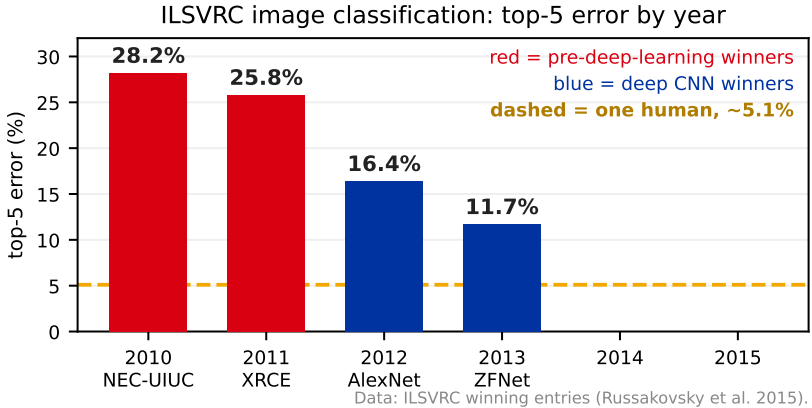
2011, XRCE (Xerox Research Europe): the same recipe with a better encoding of the hand-designed features (Fisher vectors), still linear SVMs. **Predict:** something happened in 2012. What?

One chart, one question



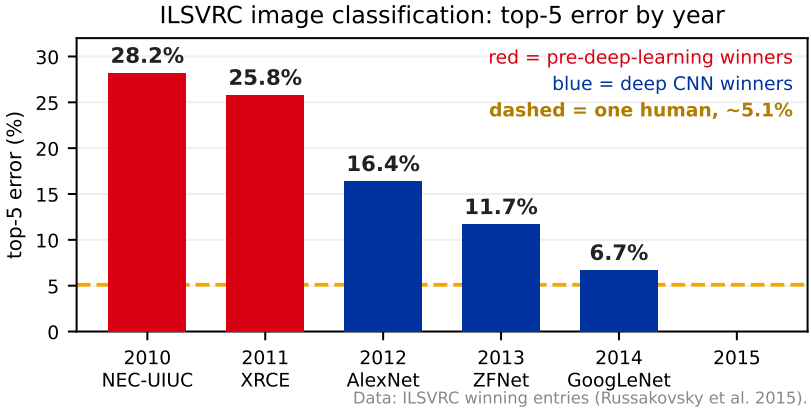
2012, AlexNet (Toronto: Krizhevsky, Sutskever, Hinton) - the first deep CNN winner: 8 learned layers, 60 million weights, two gaming GPUs. Error 26% $\rightarrow$ 16% in one year.

One chart, one question



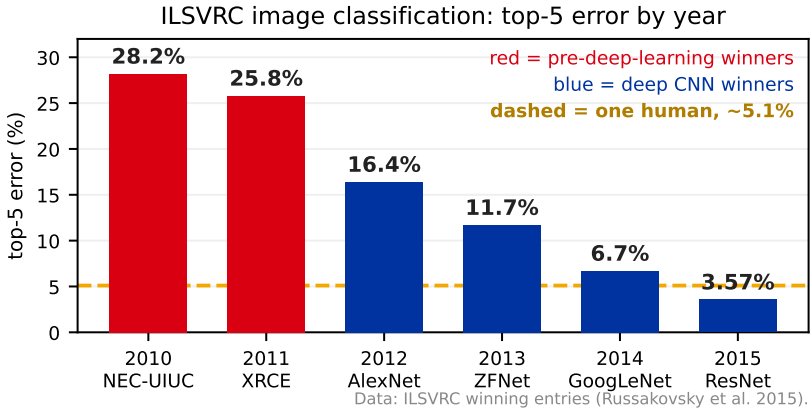
2013, ZFNet (Zeiler & Fergus, NYU; entered as Clarifai): visualized what AlexNet's layers see, then fixed what looked broken - smaller first-layer filters (7×7, stride 2).

One chart, one question



2014, GoogLeNet (Google): 22 layers of Inception modules with 12× fewer weights than AlexNet. Runner-up VGG (Oxford, 7.3%) went deep with plain 3×3 stacks.

One chart, one question



2015, ResNet (Microsoft Research Asia): 152 layers, trainable thanks to skip connections - below one human's 5.1%. **Everything in this lecture is on this chart.**

Outline

The benchmark that made deep learning

The breakout: LeNet, AlexNet, ZFNet

2014: deeper, or wider?

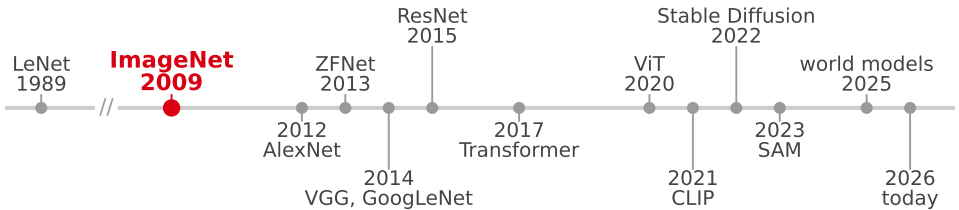
2015: ResNet

After ImageNet, and in practice

Epilogue: the field in 2026

ImageNet

Before the architectures, the arena: a dataset so large it changed what was worth building.



The bet on data

In 2006, while most of computer vision tuned **algorithms**, **Fei-Fei Li** bet that what the field actually lacked was **data**.

- Built on **WordNet**'s noun hierarchy - with WordNet's creator, Christiane Fellbaum.
- Images scraped from the web; labeled by **Amazon Mechanical Turk** workers.
- Presented at CVPR **2009**, to little fanfare: $\sim$**14M labeled images**, $\sim$22,000 categories.

The lesson that outlives every architecture in this deck: **data bets age better than algorithm bets**. Models come and go; ImageNet made all of them possible.

ILSVRC and top-5 error

The annual **ImageNet Large Scale Visual Recognition Challenge** (ILSVRC) used a subset: **1.2M training images, 1,000 classes**.

Top-5 error: the model proposes five labels; it is correct if the true label is among them - fair scoring when an image contains several things.

A shared benchmark accelerates a field for the same reason we always compare models on *held-out* data ([06]): everyone measures the same thing, so progress is real and comparable.

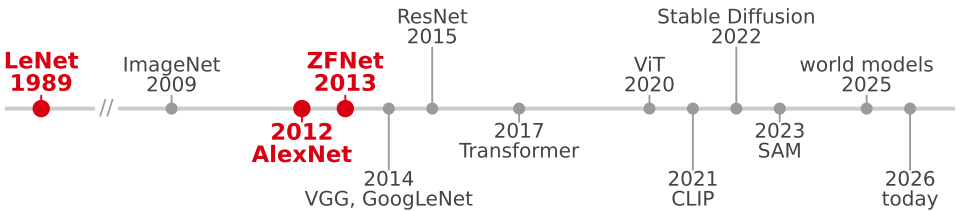


AlexNet's five guesses per photo (ILSVRC-2010).

Red bar = the correct label, if it made the top 5.

The breakout

A blueprint from 1998, two gaming GPUs, and 1.2 million photos.



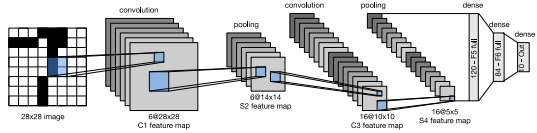
LeNet (1998): the blueprint

Yann LeCun, AT&T Bell Labs (joined 1988): backprop-trained convolutional nets reading **handwritten digits**.

`conv-pool-conv-pool-dense`: everything in [48] was already here.

Deployed commercially by NCR in **check-reading machines**; at its peak the system read an estimated **10-20% of all checks** written in the US.

Then the field moved on to SVMs, and CNNs went quiet for a decade. What was missing: **compute** and **data**.



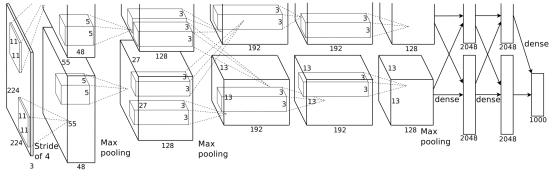
▸ play with a LeNet in 3D (Adam Harley) ▸ CNN Explainer

AlexNet (2012): the breakout

Same blueprint, **scaled**: 8 layers, **60M parameters** - too big for one GPU, so Krizhevsky **split the network across two** consumer gaming cards (GTX 580, 3GB each) and trained **5-6 days** ($\sim$90 epochs over 1.2M images) on a single PC.

The GPU was the unlock: [42]'s hardware argument, made history.

Authors: Krizhevsky, Sutskever, Hinton (2012).



The paper's Figure 2, one GPU per half - cut off at the top in the published paper too.

The "new" tricks are all **[43]-[45] material deployed at once**: ReLU instead of saturating tanh (the paper's test net hit 25% training error **6$\times$ faster**), dropout, data augmentation. You already know every ingredient - they had the hardware to use them at scale.

AlexNet: the rest of the story

- **The margin.** 15.3% top-5 error against 26.2% for the runner-up - a lead of almost 11 points in a contest usually won by one or two.
- **Home-made software.** Krizhevsky wrote his own GPU code (cuda-convnet) to fit the training into a week.
- **The auction.** Hinton and his two students formed a company, DNNresearch, and auctioned it at the NIPS conference in Lake Tahoe that December. Big tech companies bid; Hinton stopped at **$44M** and sold to Google.
- **The code.** In 2025 the Computer History Museum and Google published AlexNet's original source code on GitHub.
- **The citations.** The 8th most-cited paper of the 21st century, across all of science (Nature, 2025).

AlexNet invented almost none of its parts - they were LeNet's blueprint plus [43]-[45]. What it proved is that they **scale**.



Must watch

The moment we stopped understanding AI [AlexNet]

Welch Labs, 2024. 18 minutes: what AlexNet's layers learned, from these first-layer filters to activation atlases.

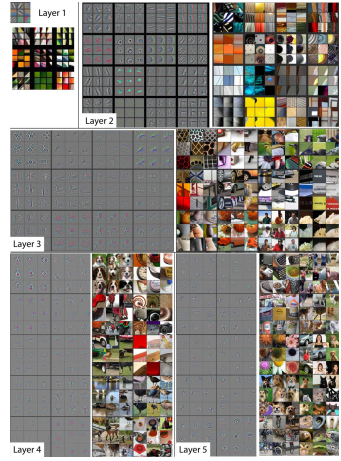
ZFNet (2013): look inside, then fix it

Matthew Zeiler and **Rob Fergus** (NYU) asked what each layer of AlexNet responds to. Their **deconvnet** runs a feature map backwards to the pixels that excited it (right: layers 1 to 5, each with the image patches that fire it most).

What they saw in AlexNet: first-layer filters that were only very coarse or very fine, some of them dead, and **aliasing** in layer 2 from the huge 11×11, stride-4 first convolution. The fix: 7×7 with stride 2 - and accuracy went up.

Zeiler's startup **Clarifai** won ILSVRC 2013 with this design: **11.7%**.

Visualization as a **debugging tool**: look at what the network learned, then change the architecture. [50]'s Grad-CAM is the same idea.



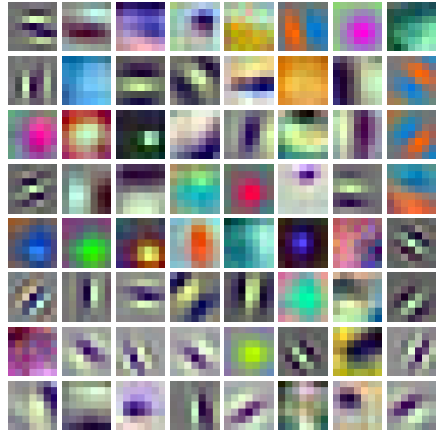
Zeiler & Fergus, ECCV 2014, Fig. 2

What AlexNet-style networks learn

First-layer filters of a modern pretrained network (ResNet-18, trained on ImageNet): crisp **Gabor-like edge, blob and color detectors**.

The **[48] kernel zoo** - reinvented by gradient descent, designed by nobody.

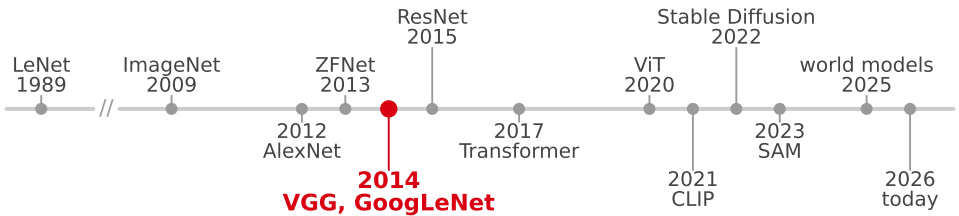
[48]'s tiny Fashion-MNIST kernels looked blurry; at ImageNet scale the learned kernels become *textbook*. The hand-designed $\rightarrow$ learned arc closes here.



ResNet-18's 64 first-layer filters (7×7, RGB)

2014: deeper, or wider?

Two answers in one year: stack small filters (VGG), or run several filter sizes side by side (GoogLeNet).

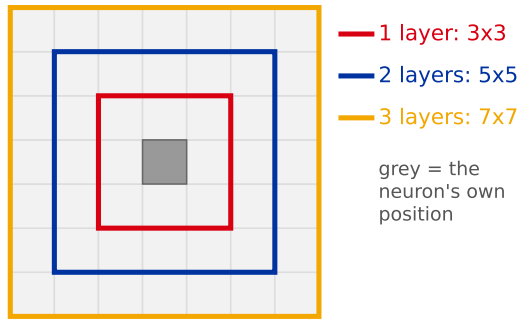


Receptive field: how far a neuron sees

The **receptive field** of a neuron is the patch of the *input image* that can influence it.

One 3×3 conv: each output sees 3×3 pixels. Put a second 3×3 conv on top: each of *its* outputs sees 3×3 outputs of the first - and together those cover 5×5 pixels. A third layer: 7×7.

Early layers see *edges*; deep layers see *objects* - [48]'s feature hierarchy, measured in pixels.



Every extra 3×3 layer adds 2 pixels of reach. (Reach *without* more layers: [48]'s dilated convolutions, back in [51].)

VGG (2014): small filters, stacked

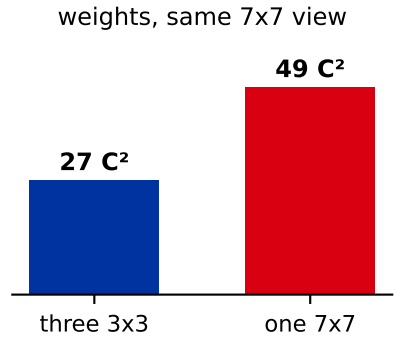
So three stacked 3×3 layers see what **one** 7×7 layer sees. **Which costs fewer weights?** (Each layer: C channels in, C out; biases ignored.)

VGG (2014): small filters, stacked

So three stacked 3×3 layers see what **one** 7×7 layer sees. **Which costs fewer weights?** (Each layer: C channels in, C out; biases ignored.)

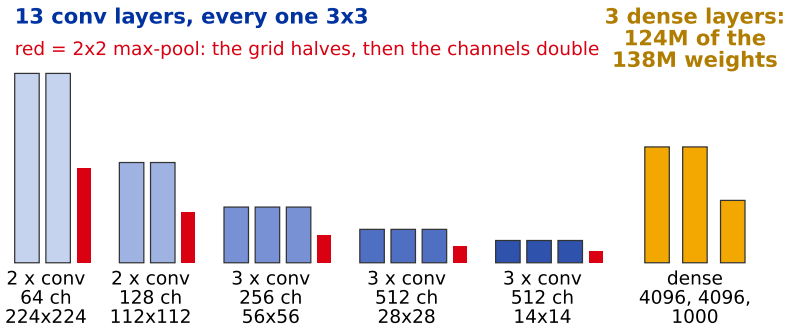
	one 7×7	**three** 3×3
sees	7×7	7×7
weights	$49\,C^2$	$3 \cdot 9\,C^2 = 27\,C^2$
at $C = 64$	200,704	110,592
nonlinearities	1 ReLU	3 ReLUs

Same view, **45% fewer weights**, and three nonlinearities instead of one. That one observation is the whole VGG design.



VGG-16, layer by layer

Repeat the trick: blocks of 3×3 convs, a 2×2 max-pool after each block, and **twice the channels** each time the grid halves.



- 2nd in 2014 (7.3%). Simple and uniform, so it became the favourite **feature extractor** - [51]'s style transfer runs on VGG.
- But the three dense layers hold **124M of the 138M** weights. Hold that thought.

Who built VGG

Karen Simonyan and Andrew Zisserman, Oxford's **V**isual **G**eometry **G**roup - and yes, Karen Simonyan is an Armenian name.

His arc since:

- **DeepMind** (2014-2022) - contributor to AlphaZero, AlphaFold, WaveNet;
- co-founder of **Inflection AI** (2022, with Reid Hoffman and Mustafa Suleyman);
- now **Chief Scientist at Microsoft AI**.

A VGG author's career *is* the last decade of AI in miniature: benchmark vision → deep-learning lab → startup → big-tech frontier lab.

1×1 convolutions

A 1×1 conv is a **per-pixel dense layer across channels**: it mixes *depth* without touching *space*.

Two jobs:

1. **cheap channel reduction** before expensive convolutions,
2. an extra **nonlinearity** almost for free.

Reduce $256 \to 64$ channels:

$$1{\times}1: \; 256 \cdot 64 + 64 = \mathbf{16{,}448} \qquad \text{vs} \qquad 3{\times}3: \; 9 \cdot 256 \cdot 64 + 64 = \mathbf{147{,}520}$$

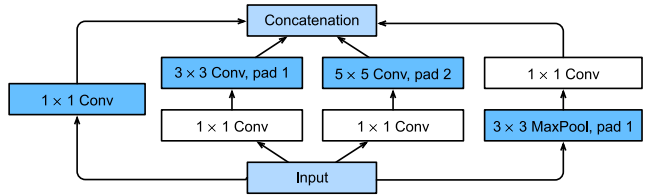
Pointwise convolutions return in [51] (separable convolutions - how phones afford CNNs).

The Inception block

Why choose a kernel size at all? Run 1×1, 3×3, 5×5 **and** a pooling path **in parallel**, and **concatenate** the maps.

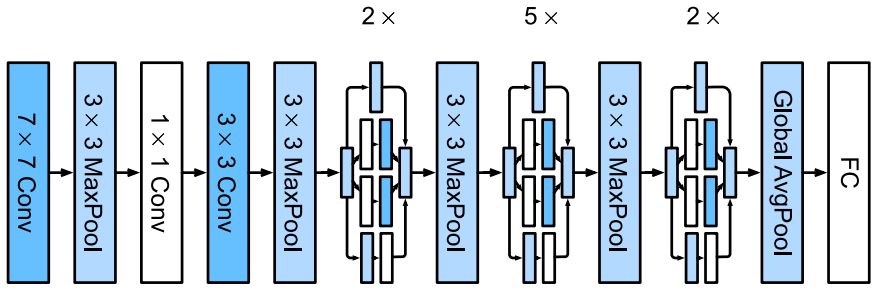
The network picks its own mix, layer by layer.

The 1×1 bottlenecks (white boxes) are what keep four parallel paths *affordable* - the previous slide's trick, deployed.



GoogLeNet (2014): a network of Inception blocks

The **2014 winner**, from Google: Inception blocks stacked **22 layers** deep - **6.7%** top-5 error with **under 7M parameters** (AlexNet: 60M).



The names. "GoogLeNet" - capital L - is an homage to LeCun's LeNet. "Inception" is the film meme "we need to go deeper"; the paper cites it as reference [1].

No giant dense head: it ends in a *global average pool* (in two slides). And it was trained on **CPUs only**, on Google's distributed system.

What GoogLeNet sees: an activation atlas

Push a **million images** through GoogLeNet and record one layer's activations (mixed4c: 512 numbers per image patch).

Lay the million vectors out in 2D so that **similar activations land close together** (UMAP, from the dimensionality-reduction chapter), average each grid cell, and render each average with feature visualization ([48]).

The result is a map of what that layer can represent: **animal faces** top left, **people** top right, **plants** on the left, **water and landscapes** at the bottom, **buildings and objects** on the right.

▸ explore it: distill.pub/2019/activation-atlas



Carter, Armstrong, Schubert, Johnson & Olah, "Activation Atlas", Distill (2019), CC BY 4.0.

The dense-head problem

Predict first: AlexNet has 60M parameters. Where do *most* of them live - the conv layers, or somewhere else?

The dense-head problem

Predict first: AlexNet has 60M parameters. Where do *most* of them live - the conv layers, or somewhere else?

In the **flatten → dense head**. AlexNet's dense layers hold roughly **58 of its 60M** weights; VGG-16's, **124M of 138M**.

Why: flattening keeps every pixel of every map. Take a modest final stack of **256 maps of** 100×100, into a 10-class dense layer:

$$256 \cdot 100 \cdot 100 \cdot 10 = \mathbf{25{,}600{,}000} \text{ weights}$$

More than all the conv layers combined - spent on one classification layer.

Global average pooling

The fix (from Network-in-Network), **global average pooling** (GAP): **average each final map to one number**. 256 maps $\rightarrow$ a 256-vector $\rightarrow$ dense-10.

Compute the head's weight count yourself.

Global average pooling

The fix (from Network-in-Network), **global average pooling** (GAP): **average each final map to one number**. 256 maps $\rightarrow$ a 256-vector $\rightarrow$ dense-10.

Compute the head's weight count yourself.

$$256 \cdot 10 = \mathbf{2{,}560} \text{ weights} \qquad (\text{was } 25{,}600{,}000) - \text{a } \mathbf{10{,}000}\times \text{ cut}$$

Bonus: GAP acts as a **regularizer** (no giant head to overfit) and accepts **any input size** (the average does not care how big the map is). Why average, not max? Global max pooling exists; the average uses every location's evidence and spreads the gradient over all of them, so it became the default.

The **[48] pooling** operation with a *global* window. Used by everything modern - ResNet already ends this way.

When averaging away *where* hurts

The chapter's practical, Gesture Snake, asks: **which way does the thumb point?** (up, down, left, right, or no thumb). A known-answer test: 1,000 synthetic crops, a disc (the fist) with a bar (the thumb) on one side. One small CNN body - three conv blocks down to 64 maps of 8×8 - with two different heads. **Predict:** does GAP still win?

When averaging away *where* hurts

The chapter's practical, Gesture Snake, asks: **which way does the thumb point?** (up, down, left, right, or no thumb). A known-answer test: 1,000 synthetic crops, a disc (the fist) with a bar (the thumb) on one side. One small CNN body - three conv blocks down to 64 maps of 8×8 - with two different heads. **Predict:** does GAP still win?

model	**held-out acc.**
small CNN + flatten head	77%
small CNN + GAP head	**25**%
pretrained ResNet-18 (ends in GAP) + linear head	99.6%
guessing, 5 classes	20%

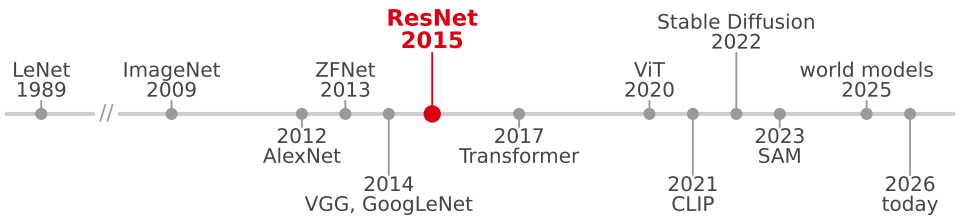
A horizontal bar looks the same pointing left or right - the answer is which *side* of the disc it sits on. This net's final features each see only about a third of the crop, so the side lives in their *position* in the 8×8 map - and GAP averages position away.

ResNet-18 averages too, yet scores 99.6%: its final features each see the **whole** crop, so "which side" is computed *before* the average. GAP is safe once the **receptive field covers the object**; before that, flatten - or go deeper.

Synthetic stand-in images, measured while building the practical (the notebook itself trains on your own photos).

If depth is good, why not 100 layers?

Because past ∼20 layers, plain networks get *worse* - and the fix reshaped every deep network since.

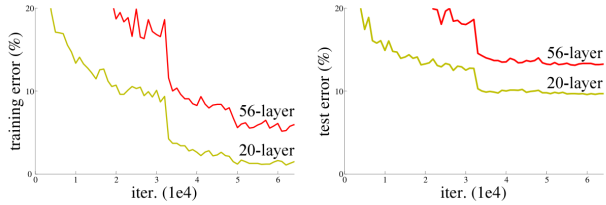


Predict first: the degradation problem

A **56-layer** plain net vs a **20-layer** plain net - same data, same recipe. **Which one has lower training error?**

Predict first: the degradation problem

A **56-layer** plain net vs a **20-layer** plain net - same data, same recipe. **Which one has lower training error?**



The 20-layer one (He et al. 2015, Figure 1). This is *not* overfitting - it is *training* error. Deep plain nets are simply **hard to optimize**: the deeper model could imitate the shallow one by making 36 layers do nothing, yet gradient descent fails to find even that. "Do nothing" is a precise target for conv+ReLU layers - every weight must line up so the input comes out unchanged, and nothing pulls a random start there.

The residual idea

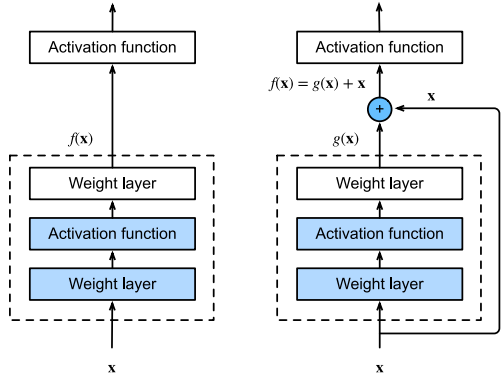
Let the block learn the **correction**, not the whole mapping:

$$H(\mathbf{x}) = F(\mathbf{x}) + \mathbf{x}$$

The **skip connection** carries $\mathbf{x}$ for free; the layers only learn the residual $F(\mathbf{x})$.

Learning "do nothing" is now trivial - push $F \to 0$ - so **extra layers can no longer hurt**.

If dimensions change, project the shortcut: $H(\mathbf{x}) = F(\mathbf{x}) + W_s\,\mathbf{x}$.



Left: a plain block must learn all of $f(\mathbf{x})$. Right: a residual block only learns the difference $g(\mathbf{x})$.

You have seen residual learning before

$$H(\mathbf{x}) = F(\mathbf{x}) + \mathbf{x}$$

Recall: where in this course have you already met a model that only fits a **correction** to what already exists?

You have seen residual learning before

$$H(\mathbf{x}) = F(\mathbf{x}) + \mathbf{x}$$

Recall: where in this course have you already met a model that only fits a **correction** to what already exists?

Gradient boosting (ch4): every new tree fits the *residual* of the ensemble so far. A ResNet block does the same thing **in depth** - boosting stacks residual learners in sequence, ResNet stacks them layer on layer.

Analogy, not identity: boosting fits each learner to the *frozen* ensemble's residual, stage by stage; ResNet blocks train *jointly*, end to end.

Why skips also fix gradients

The identity path is not just easy to learn - it is an **undamped route for the gradient** to flow backwards:

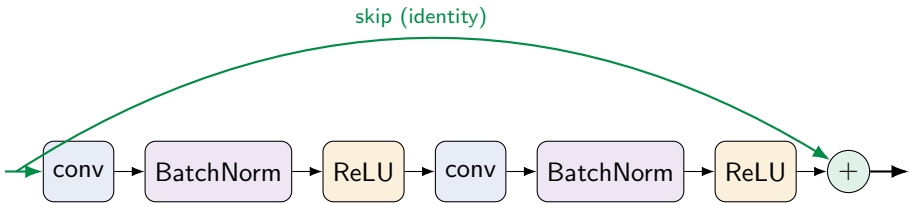
$$\frac{\partial H}{\partial \mathbf{x}} = \frac{\partial F}{\partial \mathbf{x}} + \mathbf{I}$$

In numbers (scalars, 20 layers): plain layers that each pass back half the gradient give $0.5^{20} \approx 10^{-6}$. Residual blocks pass back $1+$ their branch's share; if each branch adds ± 0.1, twenty of them give between $0.9^{20} \approx 0.12$ and $1.1^{20} \approx 6.7$ - the same order of magnitude you started with.

The **vanishing-gradient** problem from [42] and [45], relieved by *wiring*. Honest footnote: He et al. checked that their plain nets (with BatchNorm) did *not* have vanishing gradients - they were just very slow to optimize. The identity path helps both ways: a clean gradient, and "do nothing" as the easy default.

The modern conv block

What actually repeats inside a modern CNN:



`conv` → `BatchNorm` → `ReLU`, repeated, plus skip connections.

BatchNorm is the **[44] formula, unchanged** - normalize per batch, then scale and shift with learned γ, β. Nothing new to learn today.

ResNet (2015): the numbers

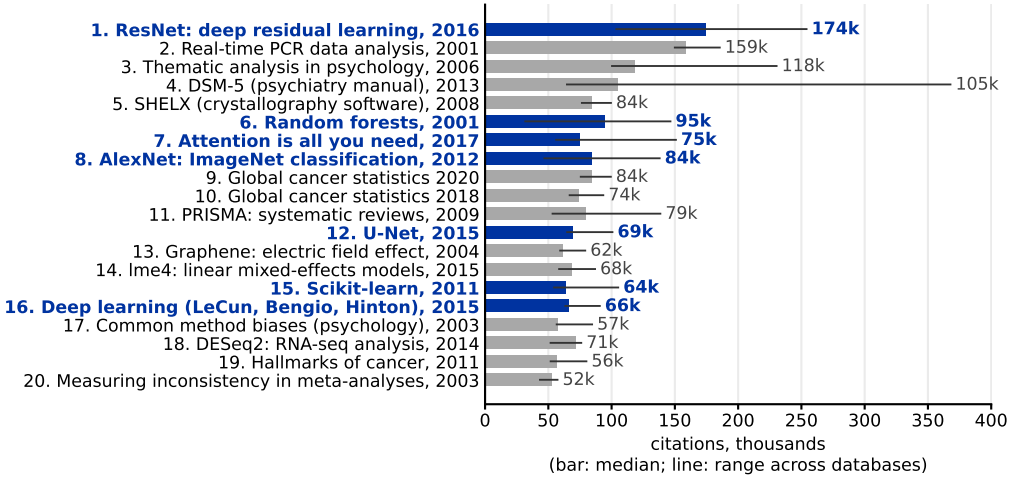
Kaiming He, Xiangyu Zhang, Shaoqing Ren and Jian Sun (Microsoft Research Asia):

- **152 layers** - 8× deeper than VGG, yet *fewer* operations (11.3 vs 15.3 billion per image).
- **3.57%** top-5 error, below the human baseline - and first place in ImageNet detection and localization and COCO segmentation, the same year.
- The **most cited paper of the 21st century**, in any field. Machine learning in Nature's top 25:

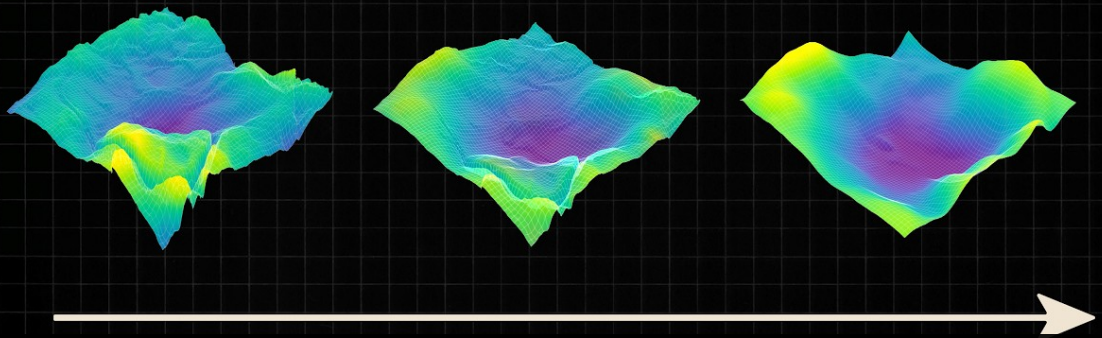
#1	Deep residual learning (ResNet), 2016	this lecture
#6	Random forests, 2001	ch4
#7	Attention is all you need, 2017	the attention chapter
#8	ImageNet classification with deep CNNs (AlexNet), 2012	this lecture
#12	U-Net, 2015	[51]
#15	Scikit-learn, 2011	every homework
#16	Deep learning (LeCun, Bengio & Hinton), 2015	

Nature, "Exclusive: the most-cited papers of the twenty-first century", April 2025.

The top 20, with their citation counts



Machine learning in **blue**: 7 of 20. Five databases disagree by up to 5.7× (Google Scholar is highest for 19 of 20), so Nature ranks by *median rank*: the bars are not sorted. Data: Nature 640, 588-592 (2025), supplementary table, March 2025.



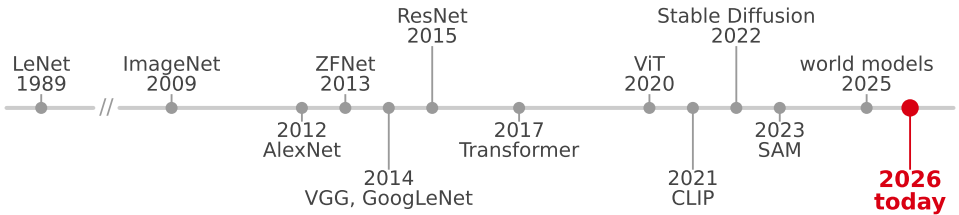
Must watch

The most cited paper of the century is a brilliant hack

Welch Labs, 2026. 35 minutes on ResNet: why deep plain networks fail to train, and why one skip connection fixes it.

After ImageNet

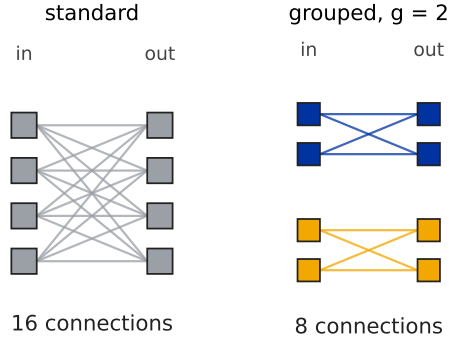
Sharper tricks, bigger searches, a saturated benchmark - and one import to get them all.



Grouped convolution

Split the input channels into g **groups**; each group gets its own filters, with no mixing across groups. Parameters and compute drop by a factor of g.

- You already saw it: **AlexNet**'s split made conv 2, 4, 5 grouped ($g = 2$), one group per card (conv 3 crossed over).
- Push it to the extreme, **one group per channel** ($g = C$), and it is the **depthwise** convolution behind separable convs ([51]).
- **ResNeXt** (2017) made g a design knob ("cardinality") - a third lever beside depth and width.



g=2 (AlexNet) to g=C (depthwise). *Deformable* conv (Dai et al., 2017) even learns *where* to sample.

Scaling and searching

Where architecture design went next - one line each:

- **DenseNet**: like ResNet, but *concatenate* the skip instead of adding it.
- **EfficientNet**: scale depth, width and resolution *together* with one compound coefficient.
- **Neural architecture search**: architectures are now *found* by search as much as designed by hand.

None of these change the vocabulary you now have: conv blocks, skips, 1×1s, GAP. The d2l.ai chapter on modern CNNs covers each, for the curious.

ImageNet today

- Top-5 error fell from 26% (2011) to 3.6% (2015). Since then papers report the stricter **top-1** accuracy: the best is about **91%** (CoCa, 2022).
- **86%** with *no* ImageNet training at all: CoCa learned from billions of image-caption pairs and is only told the 1,000 class names (CLIP-style - epilogue).
- The test set is the limit now: about **6% of ImageNet's validation labels are wrong** (Northcutt et al., 2021). Above 90%, a "mistake" is often the label's.

ImageNet is **solved enough** that it no longer ranks new models - they are compared on harder test sets (new photos, sketches, deliberately tricky images). It is still the default starting point for **transfer**: that is [50].

torchvision.models

```python
import torch
from torchvision import models

net = models.resnet18(weights="IMAGENET1K_V1")  # pretrained
print(sum(p.numel() for p in net.parameters()))  # 11,689,512

net.fc = torch.nn.Linear(512, 3)  # new head: export/market/juice
```

Load an architecture, inspect it, swap the head for *your* classes - three lines each.

What "keep the body, replace the head" buys you - and when to also retrain the body - is the whole next lecture.

▸ torchvision models docs

Which architecture, when

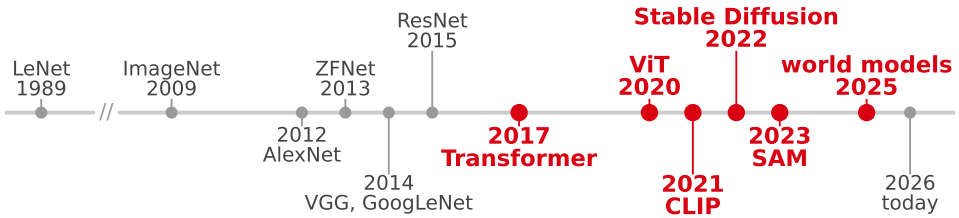
Back to the **pomegranate sorting line**:

- A Fashion-MNIST-sized toy problem (small grayscale crops, few classes) $\rightarrow$ the **small custom CNN from [48]**: 9k parameters, trains in minutes, easy to debug.
- The real sorting camera (224×224 color photos, subtle grade differences) $\rightarrow$ start from a **pretrained ResNet-family model** and adapt it.

Adapting a pretrained model to your data *is* **transfer learning** - exactly where the next lecture picks up.

Where did the field go?

Fourteen years after AlexNet: what vision looks like in 2026.



Vision in 2026 (I): understanding

Two shifts since this lecture's timeline ended:

1. **Attention ate the frontier.** Vision Transformers (ViT) treat an image as a *sequence of patches* - no convolution at all. ([51] closes this thought against CNN inductive bias.)
2. **Image-text pretraining** (Contrastive Language-Image Pre-training, CLIP, 2021) fused vision with language: today's assistants (GPT, Gemini, Claude) are **vision-language models** (VLMs) - they *see*: show them a photo, ask questions.

Classification stopped being a standalone task and became a **capability inside larger systems**.

▸ try it: show a VLM a photo, ask it questions

Vision in 2026 (II): generation and segmentation

Diffusion models flipped the task: classification asks "what is this?" - diffusion answers "*draw me one*".

- Images: Stable Diffusion, Flux, Google's Nano Banana, OpenAI's GPT Image. Video followed: Veo, Sora, Kling, Seedance.
- Intuition in one line: learn to **undo noise, step by step** - the mechanics belong to the diffusion chapter (L27-L31).
- Foundation models took the classic tasks too: **SAM 3** (Meta, 2025) - type a concept ("yellow school bus") and it segments **every** instance, in images and video, with no task-specific training.

Generative vision gets its own treatment in the **diffusion chapter** (L27-L31) - here it is context, not curriculum.

▸ SAM 3 demo: type a concept, get every mask

And yet: CNNs did not die

On phones, cars, cameras and sorting lines - including our **pomegranate belt** - CNNs and CNN hybrids remain the workhorse:

- **cheap and fast** at the edge, where there is no data center;
- the **right inductive bias** when data is limited ([48]'s whole argument);
- ResNets are still the **default baseline** in papers, a decade on;
- and given the same data and compute, convolution **keeps pace**: **ConvNeXt** (Liu et al., 2022) matched vision transformers with a pure-conv design, and DeepMind's NFNets matched them at web scale (Smith et al., 2023).

You learned the layer that still does most of the world's looking.

Recap

- A shared **benchmark** (ImageNet: 1.2M photos, 1,000 classes) turned architecture design into measurable progress.
- **AlexNet** (2012) = LeNet's blueprint + [43]-[45]'s tricks at GPU scale; **ZFNet** (2013) looked inside and fixed it.
- **VGG** (2014): stacked 3×3 filters - same view, fewer weights. **GoogLeNet** (2014): parallel filter sizes, 1×1 bottlenecks, GAP - once the receptive field covers the object.
- **ResNet** (2015): deeper plain nets *degrade* $\rightarrow$ residual learning (= boosting in depth); skips also carry gradients.
- **Load pretrained models**; don't retype architectures. ImageNet is saturated, and CNNs still run the edge.

Practical - Gesture Snake. Record your own thumb photos, then pit a small custom CNN against a pretrained ResNet-18 - and steer Snake with the winner.

Next: these models were trained on 1.2M images. You have 2,000. Transfer learning - stand on ImageNet's shoulders ([50]).